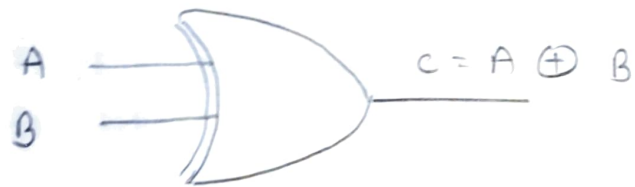


XOR GATE



OUTPUT

A	$\oplus$	B		Result
0	$\oplus$	0		0
0	$\oplus$	1		1
1	$\oplus$	0		1
1	$\oplus$	1		0

AIM: Write a C program to generate the truth table for the XOR ($\oplus$) operation.

THEORY: The XOR (exclusive OR) outputs true (1) if and only if the inputs are different, and false (0) if the inputs are same.

ALGORITHM:

1. Start
2. Read integers a and b
3. Calculate $c = a \oplus b$
4. print c
5. Stop

CODE:

```
#include <stdio.h>
```

```
int main() {
```

```
    printf("A  $\oplus$  B | Result \n");
```

```
    printf("----- \n");
```

```
    for(int A=0; A<=1; A++) {
```

```
        for(int B=0; B<=1; B++) {
```

```
            printf("%d  $\oplus$  %d | %d \n", A, B, A^B);
```

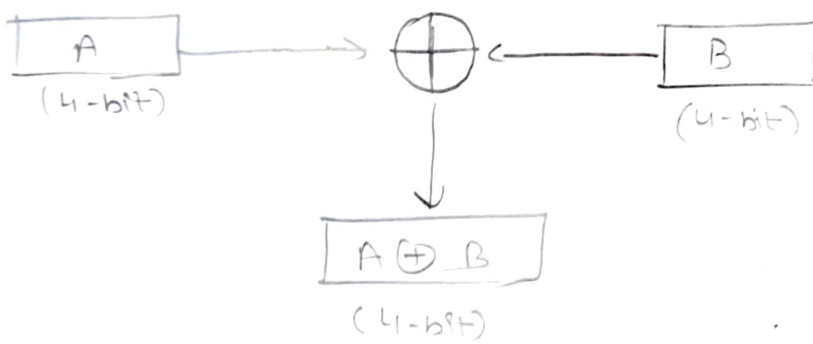
```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

DIAGRAM



AIM: Write a C program to generate the truth table for the XOR($\oplus$) operation on 4-bit binary numbers.

THEORY: XOR gives 1 when two corresponding bits are different and 0 when they are the same. This program uses the bitwise XOR operator $\wedge$ to generate the truth table for all 4-bit numbers.

ALGORITHM:

1. Start
2. Read integers a and b
3. Calculate $c = a \oplus b$
4. print/display c.
5. stop

CODE:

```
#include <stdio.h>

void print_binary (int num, int bits) {
    for (int i = bits - 1; i >= 0; i--) {
        printf ("%d", (num >> i) & 1);
    }
}

void xor_table (int bits) {
    printf ("\n A       $\oplus$       B      | Result \n");
    printf ("----- \n");

    for (int a = 0; a < (1 << bits); a++) {
        for (int b = 0; b < (1 << bits); b++) {
```

OUTPUT

<u>A</u>	<u>(+)</u>	<u>B</u>	<u>Result</u>
0000	(+)	0000	0000
0000	(+)	0001	0001
0000	(+)	0010	0010
0000	(+)	0011	0011
---	(+)	---	---
---	(+)	---	---
1111	(+)	1110	0001
1111	(+)	1111	0000

OUTPUT

<u>A</u>	<u>(+)</u>	<u>B</u>	<u>Result</u>
0000	(+)	0000	0000
0000	(+)	0001	0001
0000	(+)	0010	0010
0000	(+)	0011	0011
---	(+)	---	---
---	(+)	---	---
1111	(+)	1110	0001
1111	(+)	1111	0000

OUTPUT

<u>A</u>	<u>(+)</u>	<u>B</u>	<u>Result</u>
0000	(+)	0000	0000
0000	(+)	0001	0001
0000	(+)	0010	0010
0000	(+)	0011	0011
---	(+)	---	---
---	(+)	---	---
1111	(+)	1110	0001
1111	(+)	1111	0000

```
print_binary(a, bits);  
printf("  ⊕  " );  
print_binary(b, bits);  
printf("  |  " );
```

```
print_binary(a^b, bits);  
printf(" \n " );
```

```
}
```

```
}
```

```
}
```

```
int main() {
```

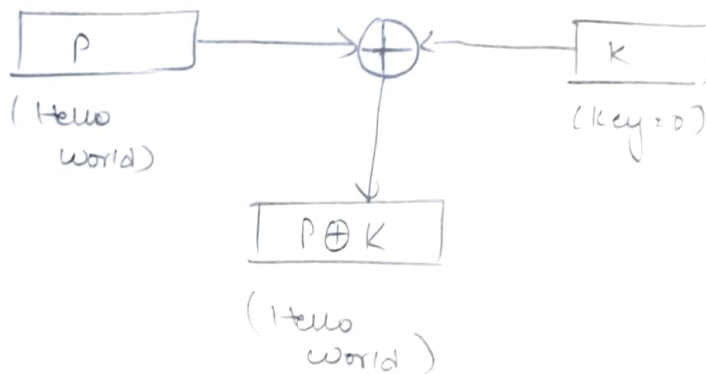
```
    int bits = 4;
```

```
    xor_table(bits);
```

```
    return 0;
```

```
}
```

DIAGRAM:

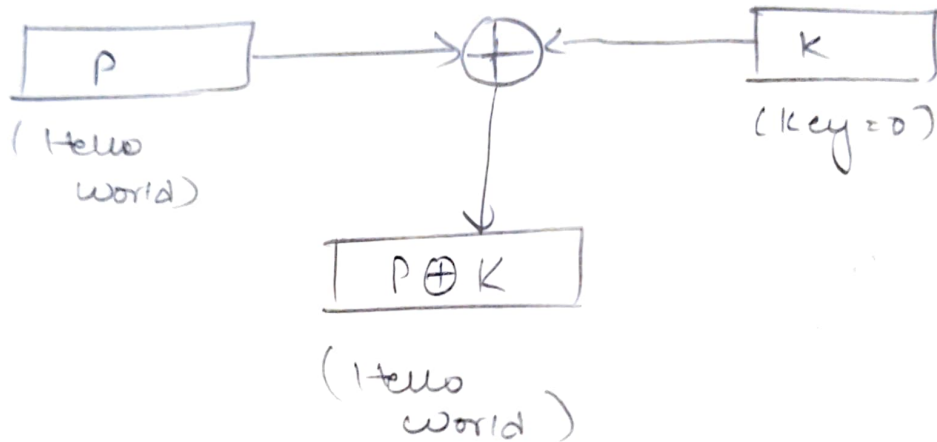


OUTPUT:

XOR with 0 results:

Character: H, XOR result: H
Character: E, XOR result: E
Character: L, XOR result: L
Character: L, XOR result: L
Character: O, XOR result: O
Character: , , XOR result: ,
Character: W, XOR result: W
Character: , , XOR result: ,
Character: R, XOR result: R
Character: L, XOR result: L
Character: D, XOR result: D

DIAGRAM:



OUTPUT:

XOR with 0 results:

Character: H, XOR result: H

Character: E, XOR result: E

Character: L, XOR result: L

Character: L, XOR result: L

Character: O, XOR result: O

Character: , XOR result:

Character: W, XOR result: W

Character: O, XOR result: O

Character: R, XOR result: R

Character: L, XOR result: L

Character: D, XOR result: D

AIM: Write a C program that contains a string (char pointer) with a value 'Hello World'. The program should XOR each character in this string with 0 and display the result.

THEORY: XORing any character with 0 leaves it unchanged. This program applies XOR with 0 to each character of "Hello World" and displays the result.

CODE:

```
#include <stdio.h>
```

```
int main() {
```

```
    char str[] = "Hello World";
```

```
    printf("Original string : %s\n", str);
```

```
    printf("XOR with 0 result : \n");
```

```
    for (int i=0; str[i] != '\0'; i++) {
```

```
        char result = str[i] ^ 0;
```

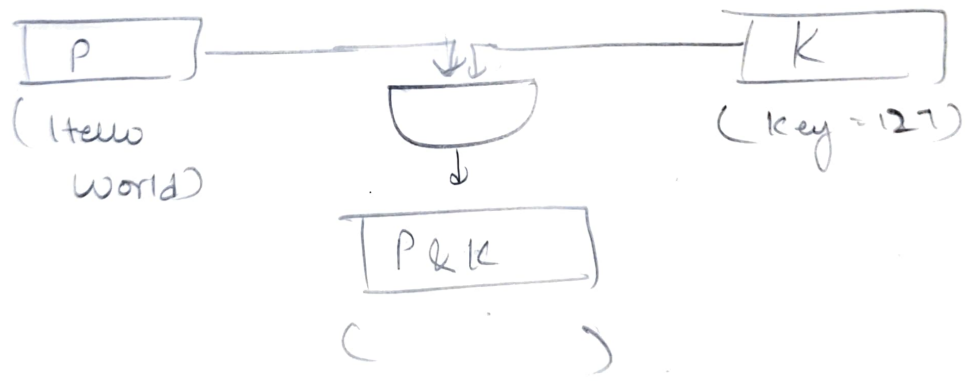
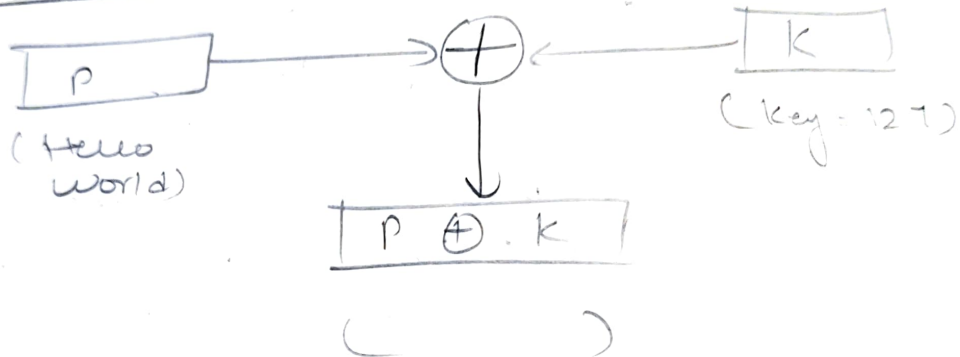
```
        printf("Character : %c, XOR result: %c\n", str[i],  
              result);
```

```
    }
```

```
    return 0;
```

```
}
```

DIAGRAM



AIM: write a C program that contains a string (char pointer) with a value 'Hello World'. The program should AND or and XOR each character in this string with 127 and display the result.

THEORY: The program applies bitwise AND and XOR operations to each character of "HELLO WORLD" using 127 (01111111 in binary). AND with 127 preserves the ASCII characters, while XOR with 127 flips their bits and displays the resulting values in hexadecimal.

CODE :

```
#include <stdio.h>
#include <string.h>
int main() {
    char *str = "Hello World";
    char and_result[50];
    char xor_result[50];

    for (int i = 0; str[i] != '\0'; i++) {
        and_result[i] = str[i] & 127;
        xor_result[i] = str[i] ^ 127;
    }

    and_result[strlen(str)] = '\0';
    xor_result[strlen(str)] = '\0';

    printf("Original string : %s\n", str);
```

OUTPUT:

Original string: Hello World

Result after AND with 127:

Hello World

Result after XOR with 127:

37 4A 13 43 10 5F 28 10 D 13 4B

```
printf("Result after AND with 127: \n");  
for (int i = 0; and-result[i] != '10'; i++) {  
    printf("%c", and-result[i]);  
}
```

```
printf("\n");
```

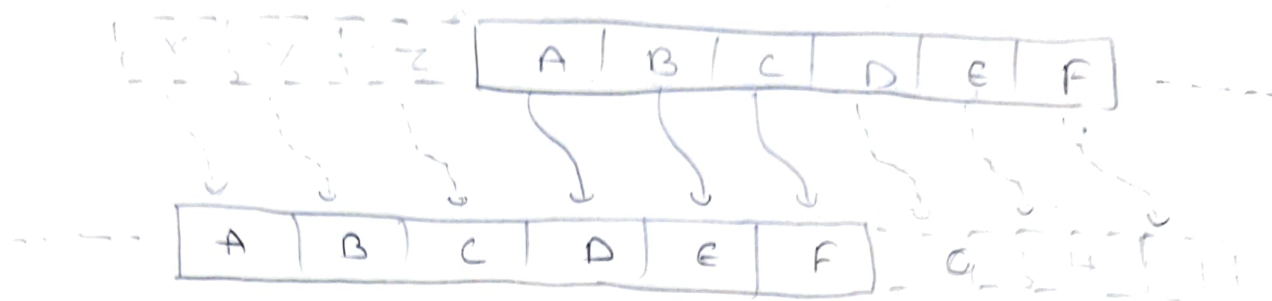
```
printf("\n Result after XOR with 127: \n");  
for (int i = 0; xor-result[i] != '10'; i++) {  
    printf(" %x", xor-result[i]);  
}
```

```
printf("\n");
```

```
return 0;
```

```
}
```

Caesar cipher structure



Substitution Cipher

A B C D E F G H I J K L M N O P Q R S T U V W X Y Z
Q W E R T Y U I O P A S D F G H J K L Z Y X V B N M

GRAY FOX HAS ARRIVED
UKQAN YAB IQL QKIOCTR

Hill cipher

Cipher text $C = K \cdot P \pmod{26}$

$$= \begin{bmatrix} 2 & 3 \\ 3 & 6 \end{bmatrix} \times \begin{bmatrix} C \\ K \end{bmatrix} \pmod{26}$$

$$= \begin{bmatrix} 2 & 3 \\ 3 & 6 \end{bmatrix} \times \begin{bmatrix} 2 \\ 10 \end{bmatrix} \pmod{26}$$

$$= \begin{bmatrix} 2 \times 2 + 3 \times 10 \\ 3 \times 2 + 6 \times 10 \end{bmatrix} \pmod{26}$$

$$= \begin{bmatrix} 34 \\ 66 \end{bmatrix} \pmod{26}$$

$$= \begin{bmatrix} 8 \\ 14 \end{bmatrix}, \text{ so corresponding alphabets are}$$

So word ATTACK became KMRIO.

AIM: Write a C program to perform encryption and decryption using the following algorithms

a. Caesar cipher. b. Substitution cipher c. Hill cipher.

THEORY:

1. Caesar Cipher.

Replaces each letter by shifting it a fixed number of positions in the alphabet. Encryption uses $C = (P + K) \bmod 26$, while decryption uses $P = (C - K + 26) \bmod 26$.

2. Substitution cipher.

Replaces each alphabet letter with another letter according to a fixed 26-character key. Decryption uses the inverse key to map each encrypted letter back to its original form.

3. Hill cipher

Converts letters into numbers from 0 to 25 and encrypts groups of letters using matrix multiplication modulo 26. $C = (K \times P) \bmod 26$. Decryption uses the modular inverse of the key matrix, which exists only when its determinant is relatively prime to 26. $P = (K^{-1} \times C) \bmod 26$

CODE:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
#define MAX 100

void caesarCipher();
void substitutionCipher();
void hillCipher();
```

Output:

Choose the encryption / decryption method:

1. Caesar cipher
2. Substitution cipher
3. Hill cipher.

Enter your choice : 1

Enter text : HELLO

Enter key (shift value) : 3

Encrypted text : KITDDR

Decrypted text : HELLO

Enter your choice : 2

Enter text : HELLO

Enter 26-character substitution key : qwerly nm

Encrypted text : PCSG

Decrypted text : HELLO

Enter your choice : 3

Enter text : HELP

Enter 2x2 key matrix:

3 3
2 5

Encrypted text : HIAT

Decrypted text : HELP

```
int main () {  
    int choice;  
    printf("Choose the encryption/decryption method:\n");  
    printf("1. Caesar cipher\n");  
    printf("2. Substitution cipher\n");  
    printf("3. Hill cipher\n");  
    printf("Enter your choice: ");  
    scanf("%d", &choice);  
    getchar();  
    switch(choice) {  
        case 1:  
            caesarCipher();  
            break;  
        case 2:  
            substitutionCipher();  
            break;  
        case 3:  
            hillCipher();  
            break;  
        default:  
            printf("Invalid choice.\n");  
    }  
    return 0;  
}  
  
void caesarCipher() {  
    char text[Max], result[Max];  
    int key, i;  
    printf("Enter text: ");
```

```
fgets(text, MAX, stdin);
text[ strlen(text, "\n") ] = '\0';
printf("Enter key (Shift value): ");
scanf("%d", &key);
for (i=0; text[i] != '\0'; i++) {
    if (isalpha(text[i])) {
        char base = isupper(text[i]) ? 'A' : 'a';
        result[i] = (text[i] - base + key) % 26 + base;
    } else {
        result[i] = text[i];
    }
}
result[i] = '\0';
printf("Encrypted text: %s\n", result);
for (i=0; result[i] != '\0'; i++) {
    if (isalpha(result[i])) {
        char base = isupper(result[i]) ? 'A' : 'a';
        text[i] = (result[i] - base - key + 26) % 26 + base;
    } else {
        text[i] = result[i];
    }
}
text[i] = '\0';
printf("Decrypted text: %s\n", text);
}

void substitutionCipher() {
    char text[MAX], encrypted[MAX], decrypted[MAX];
    char key[26], inversekey[26]; int i;
```

```

printf("Enter text: ");
fgets(text, MAX, stdin);
printf("Enter 26-charactersub key: ");
fgets(key, 27, stdin);
for (i=0; i<26; i++) {
    inversekey[key[i] - 'A'] = 'A' + i;
}
for (i=0; text[i] != '\0'; i++) {
    if (isalpha(text[i])) {
        char base = isupper(text[i]) ? 'A' : 'a';
        encrypted[i] = text[i];
    }
    encrypted[i] = '\0';
}
printf("Encrypted text: %s\n", encrypted);
for (i=0; encrypted[i] != '\0'; i++) {
    if (isalpha(encrypted[i])) {
        decrypted[i] = inversekey[encrypted[i] - base];
    } else {
        decrypted[i] = encrypted[i];
    }
}
printf("Decrypted text: %s\n", decrypted);
}

void hillCipherC() {
    int i, j, k, n=2;
    int key[2][2], inversekey[2][2], textVector[2];
    char text[MAX], encrypted[MAX], decrypted[MAX];
    printf("Enter text: ");

```

```

text[strlen(text, "\n")] = '\0';
if (strlen(text) % 2 != 0) {
    printf (" Error : Text-length must be even. \n");
    return;
}

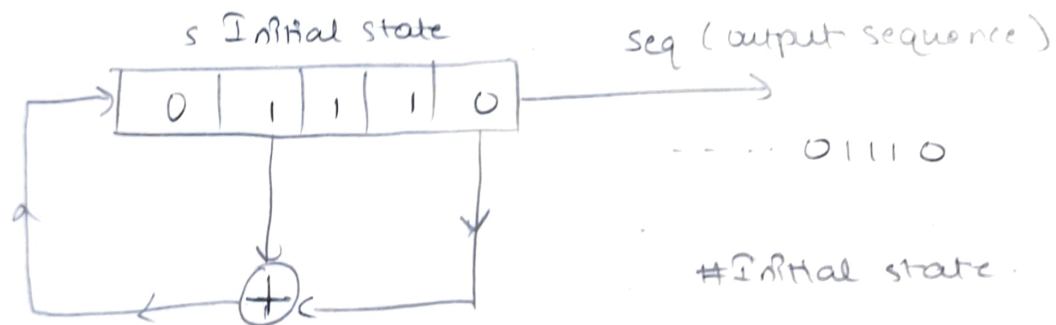
printf (" Enter 2x2 key matrix : \n");
for (i=0; i<n; i++)
    for (j=0; j<n; j++)
        scanf ("%d", &key[i][j]);
inversekey[0][0] = key[1][1] * determinantInverse % 26;
inversekey[1][0] = (-key[1][0] + 26) * determinantInverse % 26;
printf ("text : ");
for (i=0; i<strlen(text); i+=2) {
    textVector[0] = text[i] - 'A';
    textVector[1] = text[i+1] - 'A';
    for (j=0; j<n; j++) {
        resultVector[j] = 0;
        resultVector[j] += key[j][k] * textVector[k];
    }
    for (k=0; k<n; k++) {
        resultVector[j] += inversekey[j][k] * textVector[k];
    }
    resultVector[j] %= 26;
}

decrypted[i] = resultVector[0] + 'A';
decrypted[i+1] = resultVector[1] + 'A';
}

decrypted[i] = '\0';
printf ("%s\n", decrypted); }

```

LFSR



Initial state

>> s = [0, 1, 1, 1, 0]

feedback polynomial

>> fpoly = [5, 2]

AIM: Write a program on a 5-bit linear feedback shift register (LFSR) using a primitive polynomial. Primitive polynomial choice: For a 5-bit LFSR, a common primitive polynomial is: $x^5 + x^2 + 1$ which corresponds to feedback taps at bit positions 5 and 2.

THEORY: A 5-bit LFSR generates a pseudo-random binary sequence by shifting its bits and inserting a feedback bit obtained by XORing the tapped bits (bits 5 and 2), based on the primitive polynomial ($x^5 + x^2 + 1$). With a non-zero seed, it produces a maximum-length sequence of $(2^5 - 1 = 31)$ states before repeating; the all-zero seed remains stuck at zero.

CODE:

```
#include <stdio.h>
#define LFSR_SIZE 5
void print_lfsr(int lfsr[]) {
    for (int i = 0; i < LFSR_SIZE; i++) {
        printf("%d", lfsr[i]);
    }
}

void lfsr_shift(int lfsr[], int cycles) {
    printf("Cycle\tState\n");
    for (int i = 0; i < cycles; i++) {
        printf("%02d\t", i + 1);
        printf_lfsr(lfsr);
        printf("\n");
    }
}
```

OUTPUT

Cycle	State
1	00001
2	10000
3	01000
4	10100
5	01010
6	10101
7	11010
⋮	⋮
32	00001
33	10000

```
int feedback = lfsr[4] ^ lfsr[4];
```

```
for (int j = LFSR_SIZE - 1; j > 0; j--) {  
    lfsr[j] = lfsr[j-1];
```

```
}
```

```
lfsr[0] = feedback;
```

```
}
```

```
}
```

```
int main() {
```

```
    int lfsr[LFSR_SIZE] = {0, 0, 0, 0, 1};
```

```
    int cycles = 33;
```

```
    lfsr_shift(lfsr, cycles);
```

```
    return 0;
```

```
}
```

DIAGRAMS

Step-1

64-bit
plaintext
message

Step-2

Initial permutation
(IP)

LPT

RPT

Step-3

56-bit
key

16 rounds

16 rounds

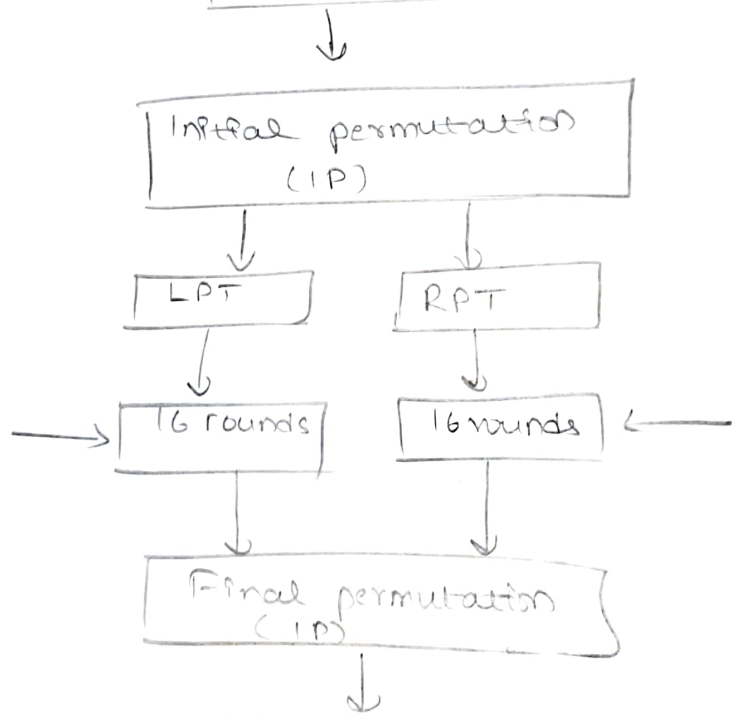
56-bit
key

Step-4

Final permutation
(IP)

Step-5

64-bit
ciphertext
message



AIM: Write a C / Java program to implement the DES Algorithm.

THEORY: Data Encryption Standard is a symmetric key block cipher used to encrypt and decrypt data. It operates on 64-bit blocks of plaintext using a 56-bit effective key. DES performs 16 rounds of encryption using permutations, substitution and XOR operations. The same secret key is used for both encryption and decryption. DES is now considered insecure because its 56-bit key can be brute-forced, so AES is preferred today.

CODE :

• java

```
import javax.crypto.Cipher;  
import javax.crypto.KeyGenerator;  
import javax.crypto.SecretKey;  
import javax.crypto.spec.SecretKeySpec;  
import java.security.SecureRandom;  
import java.util.Scanner;
```

```
class DES2 {
```

```
    static byte[] raw;
```

```
    public static void main(String[] args) {
```

```
        try {
```

```
            KeyGenerator keyGen = KeyGenerator.getInstance("DES");
```

```
            keyGen.init(56);
```

```
            SecretKey secretKey = keyGen.generateKey();
```

```
            raw = secretKey.getEncoded();
```

```
System.out.println("Generated DES symmetric key (Hex):  
    "+ bytesToHex (raw));  
Scanner scanner = new Scanner (System.in);  
System.out.print("Enter message to encrypt:");  
System.out.print("En  
byte[] inputBytes = inputMessage.getBytes();  
System.out.println("Encrypted Message (Hex):" + bytesToHex  
    (encryptedBytes));  
byte[] decryptedBytes = decrypt (raw, encryptedBytes);  
System.out.println ("Decrypted Message: " + decryptedMessage);  
} catch (Exception e) {  
    e.printStackTrace();  
}  
  
private static byte[] encrypt (byte[] key, byte[] data)  
    throws Exception {  
    SecretKeySpec keySpec = new SecretKeySpec (key, "DES");  
    Cipher cipher = Cipher.getInstance ("DES");  
    cipher.init (Cipher.ENCRYPT_MODE, keySpec);  
    return cipher.doFinal (data);  
}  
  
public static byte[] decrypt (byte[] key, byte[]  
    encryptedData) throws Exception {  
    SecretKeySpec keySpec = new SecretKeySpec (key,  
        algorithm: "DES");  
    Cipher cipher = Cipher.getInstance (Transformation: "DES");  
    cipher.init (Cipher.DECRYPT_MODE, keySpec);  
    return cipher.doFinal (encryptedData);  
}
```

OUTPUT :

Generated DES symmetric Key (Hex) : 943DAE6E97D3A008

Enter message to encrypt: HELLO WORLD

Encrypted Message (Hex) : 8A166D81E583086F230FFD5F28EFFAC

Decrypted Message: HELLO WORLD

```
public static String bytesToHex(byte[] bytes) {
```

```
    StringBuilder sb = new StringBuilder();
```

```
    for (byte b : bytes)
```

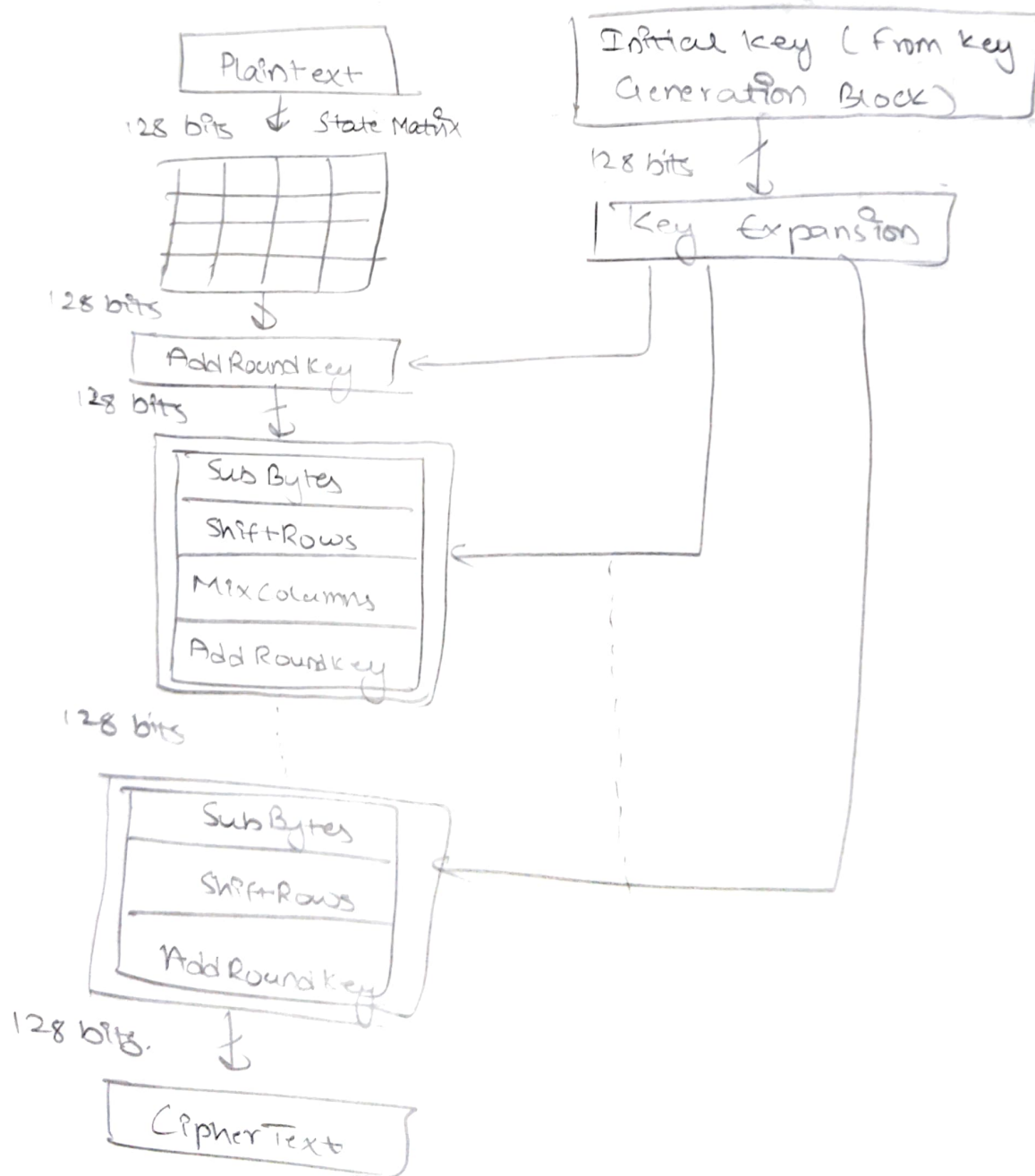
```
        sb.append(String.format("format: \"%02x\", b));
```

```
    return sb.toString();
```

```
}
```

```
}
```

DIAGRAM



AIM: Write a C program to implement the Rijndael/AES algorithm logic.

THEORY: Advanced Encryption Standard is a symmetric block cipher that encrypts data in 128-bit blocks using 128-, 192-, or 256-bit keys. It performs several rounds of substitution, permutation, and mixing operations, the same secret key is used for both encryption and decryption.

CODE:

```
import javax.crypto.*;
import javax.crypto.spec.*;
class Rijndael
{
    private static String aster(byte buf[])
    {
        StringBuffer strbuf = new StringBuffer(buf.length*2);
        int i;
        for(i=0; i<buf.length; i++){
            if(( (int) buf[i] & 0xff) < 0x10)
                strbuf.append("str: \"0\");
            strbuf.append(Long.toString((int) buf[i] & 0xff,
                16));
        }
        return strbuf.toString();
    }
    public static void main(String[] args) throws Exception
    {
        String message = "ONLINE SMART & TRAINER";
        KeyGenerator kgen = KeyGenerator.getInstance("AES");
        kgen.init(128);
```

OUTPUT :

Encrypted string: 09989624b - - - - - 98df

Original string in hexadecimal 4f4e4c - - - - - e4552

Original string: ONLINE SMART TRAINER

```
SecretKey key = kgen.generateKey();  
byte[] keyRaw = key.getEncoded();  
SecretKeySpec keySpec = new SecretKeySpec(keyRaw,  
                                             algorithm: "AES");  
Cipher cipher = Cipher.getInstance(transformation: "AES");  
cipher.init(Cipher.ENCRYPT_MODE, keySpec);  
byte[] encrypted = cipher.doFinal((args.length > 0 ?  
    message: args[0]) : getBytes());  
S.out("Encrypted string: " + asHex(encrypted));  
cipher.init(Cipher.DECRYPT_MODE, keySpec);  
byte[] original = cipher.doFinal(encrypted);  
String originalString = new String(original);  
S.out("Original string in Hexadecimal: " + asHex(original));  
S.out("Original string: " + originalString);
```

DIAGRAM

AIM: Write a C/Java program to implement the RC4 algorithm logic.

THEORY: RC4 is a symmetric stream cipher that generates a pseudo-random keystream using a secret key. It encrypts data by XORing each plaintext byte with the keystream byte, and decryption uses the same XOR operation and key to recover the original message.

CODE:

```
import javax.crypto.Cipher;
import javax.crypto.KeyGenerator;
import javax.crypto.SecretKey;
import javax.crypto.spec.SecretKeySpec;
import java.util.Scanner;

public class RC4 {
    static byte[] rawKey;

    public static void main (String [] args) {
        try {
            KeyGenerator keyGen = KeyGenerator.getInstance
                ("RC4");
            keyGen.init (keySize: 128);
            SecretKey secretKey = keyGen.generateKey();
            rawKey = secretKey.getEncoded();
            S.out ("Generated RC4 symmetric key (Hex):"
                + bytesToHex (rawKey));

            Scanner scanner = new Scanner (System.in);
            S.out (S: "Enter message to encrypt:");

            String inputMessage = scanner.nextLine();
            byte[] inputBytes = inputMessage.getBytes();
            byte[] encryptedBytes = decryptRC4 (rawKey, encryptedBytes);
            String decryptedMessage = new String (decryptedBytes);
```

OUTPUT:

Generated RC4 symmetric key (16x): AC9E37A9465-----B45B

Enter message to encrypt: HELLO

Encrypted message (16x): F7DC8D9373

Decrypted Message: HELLO

```
        sout ("Decrypted Messages:" + decryptedMessage);
    }
    catch (Exception e) {
        e.printStackTrace();
    }
}

private static byte[] encryptRC4 (byte[] key, byte[] data)
    throws Exception {
    SecretKeySpec secretKeySpec = new SecretKeySpec
        (key, algorithm: "RC4");
    Cipher cipher = Cipher.getInstance (transformation: "RC4");
    cipher.init (Cipher.DECRYPT_MODE, secretKeySpec);
    return cipher.doFinal (encryptedData);
}

public static String bytesToHex (byte[] bytes) {
    StringBuilder sb = new StringBuilder ();
    for (byte b: bytes)
        sb.append (String.format (format: "%02x", b));
    return sb.toString();
}
}
```